Notebook 01: Data Exploration

Goal: Load the RCC Visium dataset (GSE175540) and CosMx NSCLC dataset, run QC, visualize TLS signature scores on tissue sections, and sanity-check that TLS detection is working.

Run this first — before training any models.

In [1]:
```
import sys
sys.path.insert(0, '..')  # repo root

import numpy as np
import pandas as pd
import scanpy as sc
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

sc.settings.verbosity = 2
sc.settings.set_figure_params(dpi=100, facecolor='white')

print('Imports OK')
```

```
Imports OK
```

1. Load preprocessed datasets

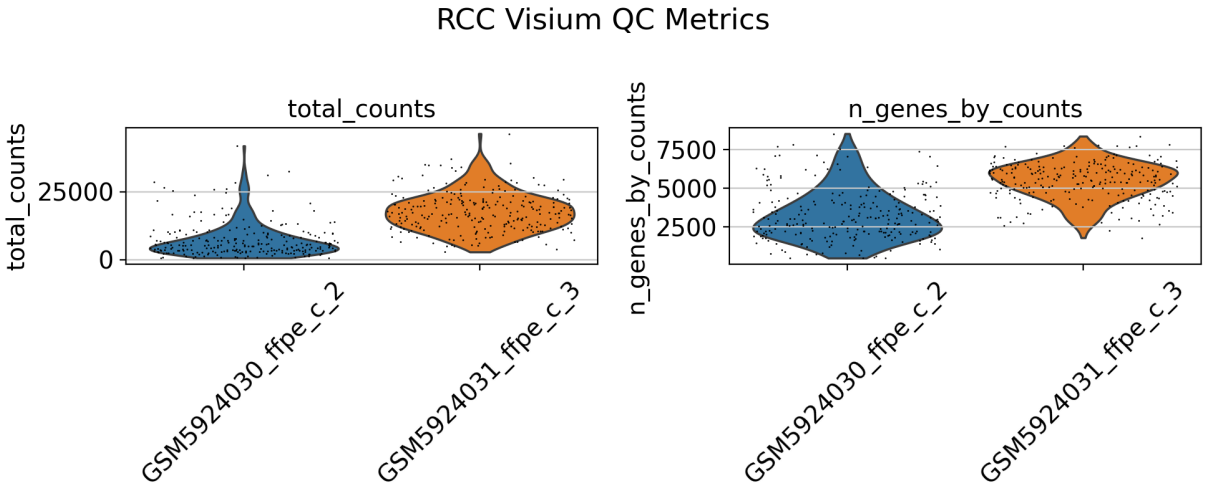
Run `python scripts/preprocess_all.py` first to generate these files.

In [2]:
```
from pathlib import Path

# Load RCC Visium
rcc_path = Path('../data/processed/rcc_visium.h5ad')
if rcc_path.exists():
    adata_rcc = sc.read_h5ad(rcc_path)
    print(f'RCC Visium: {adata_rcc}')
else:
    print(f'Not found: {rcc_path}')
    print('Run: python scripts/preprocess_all.py --dataset rcc_visium')

# Load CosMx NSCLC
cosmx_path = Path('../data/processed/cosmx_nsclc.h5ad')
if cosmx_path.exists():
    adata_cosmx = sc.read_h5ad(cosmx_path)
    print(f'CosMx NSCLC: {adata_cosmx}')
else:
    print(f'Not found: {cosmx_path}')
    print('Run: python scripts/preprocess_all.py --dataset cosmx_nsclc')
```

```
RCC Visium: AnnData object with n_obs × n_vars = 500 × 17943
    obs: 'in_tissue', 'array_row', 'array_col', 'n_genes_by_counts', 'log1p_
n_genes_by_counts', 'total_counts', 'log1p_total_counts', 'pct_counts_in_top
_50_genes', 'pct_counts_in_top_100_genes', 'pct_counts_in_top_200_genes', 'p
ct_counts_in_top_500_genes', 'n_counts', 'n_genes', 'sample_id', 'TLS_2_ca
t', 'score_cxcl13_anchor', 'score_b_cell_core', 'score_germinal_center', 'sc
ore_t_cell_zone', 'score_tfh', 'score_plasma_output', 'score_hev_markers',
'score_tls_chemokines', 'tls_composite_score', 'cxcl13_expression', 'score_t
regs', 'score_myeloid_sup', 'score_exhaustion', 'tls_candidate', 'tls_cluste
r_id'
    var: 'highly_variable', 'highly_variable_rank', 'means', 'variances', 'v
ariances_norm', 'mean', 'std'
    uns: 'hvg', 'log1p', 'pca'
    obsm: 'X_pca', 'spatial'
    varm: 'PCs'
    layers: 'counts', 'log_norm'
Not found: ../data/processed/cosmx_nsclc.h5ad
Run: python scripts/preprocess_all.py --dataset cosmx_nsclc
```

2. QC Metrics

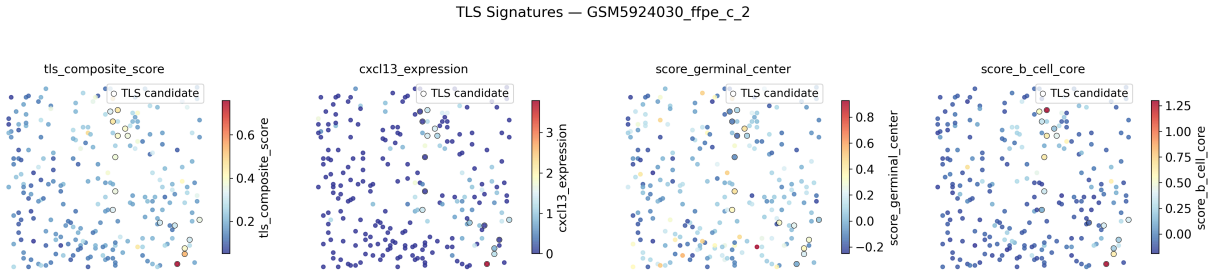
In [3]:
```
if 'adata_rcc' in dir():
    qc_cols = ['total_counts', 'n_genes_by_counts', 'pct_counts_mt']
    qc_cols = [c for c in qc_cols if c in adata_rcc.obs.columns]
    if not qc_cols:
        qc_cols = ['total_counts', 'n_genes_by_counts']

    fig, axes = plt.subplots(1, len(qc_cols), figsize=(5*len(qc_cols), 4))
    if len(qc_cols) == 1:
        axes = [axes]

    # sc.pl.violin can't accept a list of axes — call once per key
    for ax, col in zip(axes, qc_cols):
        sc.pl.violin(adata_rcc, col,
                     jitter=0.4, groupby='sample_id', rotation=45, ax=ax, sh
        ax.set_title(col)

    plt.suptitle('RCC Visium QC Metrics', y=1.02)
    plt.tight_layout()
    plt.show()
```



3. TLS Signature Scores on Tissue

In [4]:
```
from src.utils.spatial_stats import plot_spatial_score

# Show the first RCC sample
if 'adata_rcc' in dir():
    # Pick one sample with TLS
    sample_ids = adata_rcc.obs['sample_id'].unique()
    print(f'Available samples: {list(sample_ids)}')

    sample = adata_rcc[adata_rcc.obs['sample_id'] == sample_ids[0]].copy()

    score_cols = ['tls_composite_score', 'cxcl13_expression',
                  'score_germinal_center', 'score_b_cell_core']
    score_cols = [c for c in score_cols if c in sample.obs.columns]

    fig, axes = plt.subplots(1, len(score_cols), figsize=(5*len(score_cols),
    if len(score_cols) == 1:
        axes = [axes]
    for ax, col in zip(axes, score_cols):
        plot_spatial_score(sample, score_col=col, ax=ax)
    plt.suptitle(f'TLS Signatures — {sample_ids[0]}', y=1.01)
    plt.tight_layout()
    plt.show()
```

```
Available samples: ['GSM5924030_ffpe_c_2', 'GSM5924031_ffpe_c_3']
```



4. TLS Candidate Regions

In [5]:
```
if 'adata_rcc' in dir():
    if 'tls_candidate' in adata_rcc.obs.columns:
        n_tls = adata_rcc.obs['tls_candidate'].sum()
        n_clusters = adata_rcc.obs['tls_cluster_id'].nunique() - 1  # exclud
        print(f'TLS candidates: {n_tls} spots across {n_clusters} clusters')
        print(adata_rcc.obs.groupby('sample_id')['tls_candidate'].sum())
    else:
        print('TLS detection not yet run. Run preprocess_all.py first.')
```

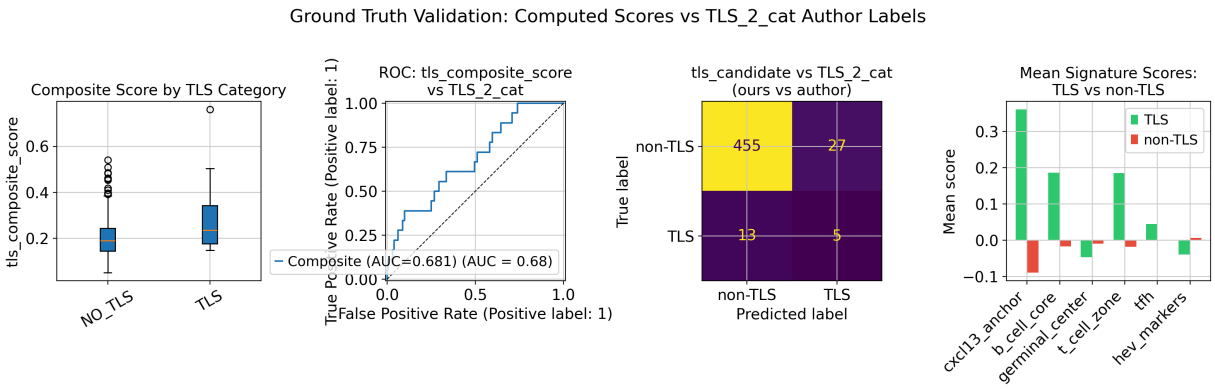
```
TLS candidates: 32 spots across 8 clusters
sample_id
GSM5924030_ffpe_c_2    16
GSM5924031_ffpe_c_3    16
Name: tls_candidate, dtype: int64
```

4b. Sanity Check: Computed Scores vs Author Ground Truth (`TLS_2_cat`)

In [6]:
```
from sklearn.metrics import roc_auc_score, confusion_matrix, ConfusionMatrix
import matplotlib.gridspec as gridspec

if 'adata_rcc' in dir() and 'TLS_2_cat' in adata_rcc.obs.columns:
    obs = adata_rcc.obs.copy()

    # TLS_2_cat is Categorical — convert to plain string for safe comparison
    tls_cat = obs['TLS_2_cat'].astype(str)   # NaN becomes 'nan'

    # --- 1. Label distribution ---
    print("Author TLS_2_cat label counts:")
    print(tls_cat.value_counts(dropna=False))
    print()

    # Negative label is the most common category (NO_TLS)
    neg_label = tls_cat.value_counts().idxmax()
    binary_label = (tls_cat != neg_label).astype(int)
    print(f"Negative class: '{neg_label}' | TLS-positive spots: {binary_labe
    print()

    cats = tls_cat.unique()

    fig = plt.figure(figsize=(16, 5))
    gs = gridspec.GridSpec(1, 4, figure=fig)

    # Box plot: tls_composite_score by TLS category
    ax0 = fig.add_subplot(gs[0])
    score_by_cat = [obs.loc[tls_cat == c, 'tls_composite_score'].values for
    ax0.boxplot(score_by_cat, tick_labels=cats, patch_artist=True)
    ax0.set_title('Composite Score by TLS Category')
    ax0.set_ylabel('tls_composite_score')
    ax0.tick_params(axis='x', rotation=30)

    # ROC curve
    ax1 = fig.add_subplot(gs[1])
```

```
    n_pos = binary_label.sum()
    if 0 < n_pos < len(binary_label):
        auc = roc_auc_score(binary_label, obs['tls_composite_score'])
        RocCurveDisplay.from_predictions(binary_label, obs['tls_composite_sc
                                         ax=ax1, name=f'Composite (AUC={auc:
        ax1.plot([0, 1], [0, 1], 'k--', lw=0.8)
        print(f"AUROC vs TLS_2_cat: {auc:.3f}")
    else:
        ax1.text(0.5, 0.5, f'n_pos={n_pos}\ncannot compute ROC', ha='center'
    ax1.set_title('ROC: tls_composite_score\nvs TLS_2_cat')

    # Confusion matrix: our tls_candidate flag vs author label
    ax2 = fig.add_subplot(gs[2])
    if 'tls_candidate' in obs.columns:
        cm = confusion_matrix(binary_label, obs['tls_candidate'].astype(int)
        ConfusionMatrixDisplay(cm, display_labels=['non-TLS', 'TLS']).plot(a
        ax2.set_title('tls_candidate vs TLS_2_cat\n(ours vs author)')
        prec = cm[1,1] / (cm[0,1] + cm[1,1] + 1e-9)
        rec  = cm[1,1] / (cm[1,0] + cm[1,1] + 1e-9)
        print(f"tls_candidate  precision={prec:.3f}  recall={rec:.3f}")

    # Bar chart: mean signature scores TLS vs non-TLS
    ax3 = fig.add_subplot(gs[3])
    score_cols = ['score_cxcl13_anchor', 'score_b_cell_core', 'score_germina
                  'score_t_cell_zone', 'score_tfh', 'score_hev_markers']
    score_cols = [c for c in score_cols if c in obs.columns]
    tls_means    = obs.loc[binary_label == 1, score_cols].mean()
    nontls_means = obs.loc[binary_label == 0, score_cols].mean()
    x = np.arange(len(score_cols))
    w = 0.35
    ax3.bar(x - w/2, tls_means.values,    w, label='TLS',     color='#2ecc71
    ax3.bar(x + w/2, nontls_means.values, w, label='non-TLS', color='#e74c3c
    ax3.set_xticks(x)
    ax3.set_xticklabels([c.replace('score_', '') for c in score_cols], rotat
    ax3.set_title('Mean Signature Scores:\nTLS vs non-TLS')
    ax3.set_ylabel('Mean score')
    ax3.legend()

    plt.suptitle('Ground Truth Validation: Computed Scores vs TLS_2_cat Auth
    plt.tight_layout()
    plt.show()
else:
    print('TLS_2_cat column not found — check that preprocess_all.py loaded
```

```
Author TLS_2_cat label counts:
TLS_2_cat
NO_TLS    482
TLS        18
Name: count, dtype: int64

Negative class: 'NO_TLS' | TLS-positive spots: 18/500

AUROC vs TLS_2_cat: 0.681
tls_candidate  precision=0.156  recall=0.278
```



5. CosMx: Cell Type Composition Within TLS Regions

In [7]:
```
if 'adata_cosmx' in dir() and 'cell_type' in adata_cosmx.obs.columns:
    # Compare cell type composition: TLS vs non-TLS regions
    tls_mask = adata_cosmx.obs.get('tls_candidate', pd.Series(False, index=a

    tls_comp = adata_cosmx[tls_mask].obs['cell_type'].value_counts(normalize
    non_tls  = adata_cosmx[~tls_mask].obs['cell_type'].value_counts(normaliz

    comp_df = pd.concat([tls_comp, non_tls], axis=1).fillna(0)
    comp_df.sort_values('TLS', ascending=False).head(15).plot(
        kind='bar', figsize=(12, 5), title='Cell Type Composition: TLS vs no
    )
    plt.ylabel('Fraction')
    plt.tight_layout()
    plt.show()
else:
    print('CosMx data or cell_type annotations not available.')
```

```
CosMx data or cell_type annotations not available.
```

6. Functional State Label Distribution (CosMx)

In [8]:
```
if 'adata_cosmx' in dir() and 'functional_state_name' in adata_cosmx.obs.col
    label_counts = adata_cosmx.obs[
        adata_cosmx.obs['functional_state_name'] != 'non_tls'
    ]['functional_state_name'].value_counts()

    print('TLS Functional State Labels (CosMx):')
    print(label_counts)

    label_counts.plot(kind='bar', color=['#2ecc71', '#e74c3c', '#95a5a6'],
                      title='TLS Functional State Labels')
    plt.ylabel('Number of TLS regions')
    plt.tight_layout()
    plt.show()
else:
    print('Functional state labels not yet generated.')
```

```
Functional state labels not yet generated.
```

Next steps

1. If TLS detection looks reasonable → run `notebooks/02_tls_detection.ipynb` to tune thresholds
2. If CosMx labels look reasonable → run `notebooks/03_pseudo_labels.ipynb` to validate labels
3. Proceed to graph construction → `notebooks/04_graph_construction.ipynb`